

IsA and HasA Relationship in C# with Examples

In this article, I will discuss **IsA and HasA Relationships in C#** with Examples. Please read our [Inheritance in C#](#) article before proceeding to this article. It is very important to understand the concept of IS-A and HAS-A relationship between classes. Many students and developers, especially beginners and freshers, get confused between the IS-A and HAS-A relationship. Because of this, they are making the relationship between classes in the wrong order.

So, today, I will explain what IS-A and HAS-A relationships are exactly, what the differences between them are, and when and how to use IS-A and HAS-A relationships in real-time applications. Here, I will show you examples using the C# language, which also applies to any other object-oriented programming language, such as Java, C++, etc.

IsA and HasA Relationship in C#

In C#, like in many other object-oriented programming languages, the concepts of “IsA” and “HasA” relationships are fundamental to understanding inheritance and composition, respectively. These relationships help design the structure of classes and objects in a program.

IsA Relationship (Inheritance):

Definition: The IsA relationship is achieved through inheritance. It represents a hierarchical relationship between a base class (parent) and derived classes (children). In an IsA relationship, the derived class is a specialized version of the base class.

Characteristics:

- **Inheritance:** The derived class inherits properties and methods from the base class.
- **Polymorphism:** The derived class can override or extend the functionalities of the base class.
- **Substitutability:** Objects of the derived class can be treated as base class objects.

HasA Relationship (Composition):

Definition: The HasA relationship is used to denote usage or composition. It indicates that an object of one class “contains” or “is composed of” objects from another class. This relationship is less tightly coupled than inheritance.

Characteristics:

- **Composition:** A class contains or is composed of objects from another class.
- **Independence:** The contained object (e.g., Engine) can exist independently of the container (e.g., Car).
- **Encapsulation:** The internal workings of the contained object are usually hidden from the outside class.
- **Flexibility and Reusability:** Objects can be easily replaced or changed, providing more flexibility in code design.

IsA and HasA Relationship in C# with Examples:

Let us understand the IS-A and HAS-A Relationship in C# with one example. Please have a look at the example below.

```
class Rectangle{  
    Some Data Members  
    Some Member Functions  
}
```

This is a class called Rectangle. Let us assume we have some data members and member functions there inside this class. Next, we have a class called Cuboid, which is inheriting from the above Rectangle class as follows.

```
class Cuboid : Rectangle{  
    Some Data Members  
    Some Member Functions  
}
```

This is the Cuboid class, which is inherited from the Rectangle class. Also, assume that this class also has some data members and member functions inside it. Now let us write one more class as follows.

```
class Table{  
    Rectangle top;  
    int legs;  
}
```

This is the Table class, which is not inherited from any class. This class has two data members. One of the data members is of class Rectangle, and the other is of integer type variable, i.e., top and legs.

So first, we created a class called Rectangle. Then we created another class called Cuboid, which is inherited from the Rectangle class, and then we created another class called Table. Inside the Table class, we created one variable of the type Rectangle and another of integer type.

The class **Cuboid** is inherited from the **Rectangle** class. So, can we say that a **Cuboid IS A Rectangle? Yes**. So, the relationship between the Rectangle and Cuboid classes is the **IS-A** relationship. So, the point you need to remember is when we are making the Inheritance or Parent-Child relationship, we can say that relationship is an IS-A relationship in object-oriented programming languages.

Next, our Table class has a top variable that is of the Rectangular type. That means the **Table** class has an object of the Rectangle class. So, can we say that the **Table class HAS A Rectangle? Yes**, the Table class Has a Rectangle. So, the relationship between the Table and the Rectangle class is the **Has A** relationship.

So, we can use our class in two ways: **Is A** and **Has A**. This is common in object-oriented programming languages, just not for C#; it is available in C++, Java, and other Object-Oriented Programming languages.

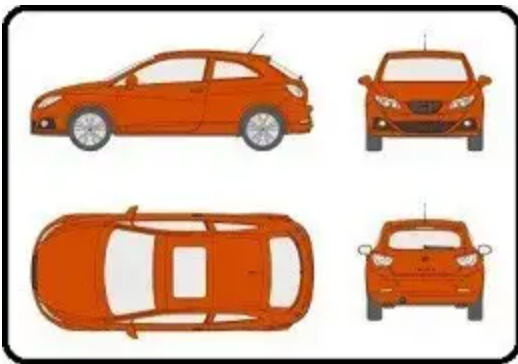
So, a class in C# can be used in two ways. One way is by using the inheritance relationship, i.e., one class inherits from that class. The second approach is that we can create an object of that class and use it. So, there are two ways of using one class. You can create and use the object or inherit it from that class.

Real-time Example to Understand Is-A and Has-A Relationship:

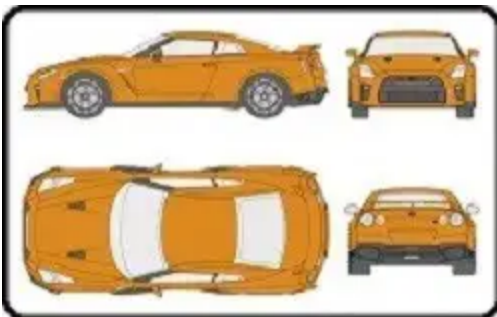
Let us understand **IS-A** and **HAS-A** relationships with one Real-time Example. Let us take an example. Please have a look at the below image.



Let us assume that the above image shows the design of my car. Now I can manufacture any number of cars from the above design and I can sell them on the market. Let us assume we have manufactured the following cars from the above design.



Manufacturing the Cars means these are objects. Now, I can sell my design (not the manufactured cars or objects, only the design) to your company, and you can manufacture your car with some changes in the above design, which I will sell to you.



So, what are the uses of design class?

What does the use of the design class mean? In how many ways can we use the design class? We can use the design class in two ways. Either we share it with the derived classes or create an object. If a class inherits from another class, it has an 'IS A' relationship with that class, or if a class has an object of some other class, then it has a 'HAS A' relationship with that class. So, a **Table has a Rectangle**, and a **Cuboid is a Rectangle**.

Next, the important thing is that a class can have different types of members, i.e., we have six types of members: public, private, protected, internal, protected, and private. But let us assume that we have three types of members in our class: **private**, **protected**, and **public**.

Now, when you have those three types of members, then which members are accessible inside the class, which members are accessible in the class that is an 'IS A' relationship, and which members are accessible in the class that has a 'HAS A' relationship? So, it means what is accessible inside the class, what is accessible in derived classes, and what is accessible upon an object. We already discussed this in our Access Specifiers article. All the members are accessible within the class, public and protected members are accessible in the derived classes (IS-A relationship), and public members are only accessible from non-derived classes (HAS-A relationship).

Example to Understand IS-A Relationship in C#

Please have a look at the example below. In the below example, the relation between Rectangle and Cuboid is IS-A relationship, so we can say that Cuboid IS-A Rectangle.

```
using System;
namespace IsAHasADemo
{
    public class Program
    {
        static void Main()
        {
            Cuboid cuboid = new Cuboid(3, 5, 7);
            Console.WriteLine($"Volume is : {cuboid.Volume()}");
            Console.WriteLine($"Area is : {cuboid.Area()}");
            Console.WriteLine($"Perimeter is : {cuboid.Perimeter()}");
            Console.ReadKey();
        }
    }
    class Rectangle
    {
        //Data Members
        public int Length;
        public int Breadth;
        //Member Functions
        public int Area()
        {
            return Length * Breadth;
        }
        public int Perimeter()
        {
            return 2 * (Length + Breadth);
        }
    }
    //Establishing Parent-Child Relationship
    //IS-A Relationship i.e. Cuboid IS-A Rectangle
    class Cuboid : Rectangle
    {
        public int Height;
        public Cuboid(int l, int b, int h)
        {
            Length = l;
            Breadth = b;
        }
    }
}
```

```

        Height = h;
    }
    public int Volume()
    {
        return Length * Breadth * Height;
    }
}
}

```

Output:

```

Volume is : 105
Area is : 15
Perimeter is : 16

```

Example to Understand HAS-A Relationship in C#

Please have a look at the example below. In the example below, within the Employee class, we have created one variable of Address type, which is nothing but a HAS-A relationship between Employee and Address. So, we can say that Employee HAS-A Address.

```

using System;
namespace IsAHasADemo
{
    public class Program
    {
        static void Main()
        {
            Address address = new Address("B1-3029", "BBSR", "Odisha");
            Employee employee = new Employee(1001, "Ramesh", address);
            employee.Display();
            Console.ReadKey();
        }
    }
    class Address
    {
        public string AddressLine, City, State;
        public Address(string addressLine, string city, string state)
        {
            AddressLine = addressLine;
            City = city;
            State = state;
        }
    }
    class Employee
    {
        //Using Address in Employee class
        //Establishing Has-A relationship i.e. Employee HAS-A Address
        public Address address;
    }
}

```

```

public int Id;
public string Name;
public Employee(int id, string name, Address adrs)
{
    Id = id;
    Name = name;
    address = adrs;
}
public void Display()
{
    Console.WriteLine($"Employee Id: {Id}");
    Console.WriteLine($"Employee Name: {Name}");
    Console.WriteLine($"AddressLine: {address.AddressLine}");
    Console.WriteLine($"City: {address.City}");
    Console.WriteLine($"State: {address.State}");
}
}
}

```

Output:

```

Employee Id: 1001
Employee Name: Ramesh
AddressLine: B1-3029
City: BBSR
State: Odisha

```

In the above example, if we inherit the Employee Class from the Address class, then it does not make any sense. If we inherit, it will establish an IS-A relationship between Employee and Address classes, and it does not make sense to say Employee IS-A Address. Rather, if we implement a HAS-A relationship, then it makes sense to say Employee HAS-A Address.

Differences Between IsA and HasA Relationship in C#

- **Nature of Relationship:** IsA is an inheritance (parent-child) relationship, whereas HasA is a compositional relationship.
- **Coupling:** IsA typically implies a stronger coupling between classes, as the derived class is tightly linked to its base class. HasA implies a looser coupling, as the objects can exist independently.
- **Usage:** IsA is used when a class needs to be an extension of another class, inheriting its properties and behaviors. HasA is used when a class requires certain functionalities or properties that are encapsulated in another class.
- **Flexibility:** Composition (HasA) offers more flexibility as components can be easily replaced or modified without affecting the container class

How do you decide what to implement between IS-A and HAS-A?

Ask the question yourself. For example, if I ask you the question, which statement gives you more sense from the two statements below?

Employee IS-A Address
Employee HAS-A Address

Then definitely, you will tell Employee HAS-A Address gives more sense than Employee IS-A Address. If I ask you, which statement gives you a better sense of the two statements?

BMW IS-A Car
BMW HAS-A Car

Then, your answer will be definitely BMW IS-A Car.

Note: In C#, the IS-A relationship is implemented using Inheritance, and the HAS-A relationship is implemented using Composition, i.e., declaring a variable. So, whenever we declare a variable of one class inside another class, we call it a Composition, or you can say HAS-A relationship.

In the next article, I will discuss [Generalization and Specialization in C#](#) with Examples. In this article, I try to explain **the IsA and HasA Relationships in C# with examples, and I hope you enjoy this IS-A and HAS-A relationship** in C# article. I would like to have your feedback. Please post your feedback, questions, or comments about this article.